BOLETÍN PRÁCTICA 1
Programación Funcional en LISP

OBJETIVOS



Tras la realización de este boletín, y del estudio de la parte correspondiente de la asignatura, deberás ser capaz de:

- Implementar funciones en LISP.

TEMÁTICA

Tras haber estudiado el lenguaje LISP, en esta práctica se trata de practicar la programación en el lenguaje mediante la realización de programas de mayor dificultad a los realizados en el aula.

TRABAJO PREVIO

- Diseño de las estrategias que se utilizarán para resolver el problema.

La sesión de prácticas se dedicará a implementar el enfoque, y no a diseñar las estrategias necesarias para la resolución del ejercicio.

ENUNCIADO

Ejercicio 1

Realiza las siguientes actividades en LISP, utilizando un enfoque puramente funcional (sin bucles, setq…). Cada función deberá tener un comentario que explique para qué sirve

1. Implementar una función llamada **producto_lista** que calcule el producto de una lista de números. En el caso de una lista vacía, deberá devolver el elemento neutro de la multiplicación (1).

2. Definir una función llamada **borrar_n_esimo** que a partir de una lista y un entero positivo devuelva otra lista sin el elemento que ocupa la posición especificada mediante el entero.

3. Implementar una función llamada **aplanar** que a partir de una lista anidada, retorne una lista sin anidamientos que contenga a todos los átomos.

4. Definir una función llamada **aplicar** que tome como argumentos un número entero positivo *n* y una función unaria f(*x*) y devuelva

$$\sum_{i=1}^{n} f(i)$$

5. Define una función llamada **filtrar** que tome como argumentos una lista sin anidamientos y una función de un argumento que devuelva un booleano como resultado y devuelva como resultado una lista con los elementos para los que la función de filtrado devuelva T.

6. Define una función llamada **listar** que emule el comportamiento de la función LIST. La función debe admitir un número arbitrario de parámetros y devolver una lista con los parámetros

Ejercicio 2

Implementa una función llamada **aplicarFunciones** que reciba como parámetros una lista y dos funciones unarias, y devuelva el resultado de aplicar la primera función sobre los elementos que ocupan posiciones impares y la segunda sobre los que ocupan posiciones pares.

Ejemplo:
> (aplicarFunciones (lambda(x) (1+ x)) (lambda(x) (1- x)) '(1 2 3 4 5))
(2 1 4 3 6)

Repite el ejercicio anterior, pero pasando los elementos en forma de número arbitrario de parámetros en lugar de una lista. Llama a la función **aplicarFunciones2**

Ejemplo:
> (aplicarFunciones2 (lambda(x) (1+ x)) (lambda(x) (1- x)) 1 2 3 4 5)
(2 1 4 3 6)

Ejercicio 3

Definir una función llamada **subconjuntos** que a partir de una lista genere todos los posibles subconjuntos que puedan formase con sus elementos. Por ejemplo:

> (subconjuntos '(a b))
((a b) (a) (b) ())
> (subconjuntos '(1 2 3))
((1 2 3) (1 2) (1 3) (1) (2 3) (2) (3) NIL)

Contempla dividir la solución en varias funciones

Ejercicio 4

Una forma de representar matrices es como una lista de listas, donde cada fila viene representada por una lista de los elementos que la forman. Implementar una función llamada **transpuesta** que dada una matriz M expresada en este formato devuelva su transpuesta. Como en el caso anterior, contempla el uso de varias funciones.

Ejemplos:
> (transpuesta '((1 2 3 4) (5 6 7 8))
((1 5) (2 6) (3 7) (4 8))
> (transpuesta (transpuesta '((1 2 3) (4 5 6))))
((1 2 3) (4 5 6))